

Sistemi di Elaborazione Accelerata

# **Trasformata Discreta di Fourier per Edge Detection su Immagini**

Francesco Moretti  
Giacomo Sampaoli

# Indice

|           |                                                                |           |
|-----------|----------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Studio del problema</b>                                     | <b>2</b>  |
| 1.1       | DFT e IDFT . . . . .                                           | 2         |
| 1.2       | Filtro . . . . .                                               | 2         |
| 1.3       | Formato immagine . . . . .                                     | 3         |
| <b>2</b>  | <b>Implementazione Sequenziale Naïve</b>                       | <b>4</b>  |
| 2.1       | DFT . . . . .                                                  | 4         |
| 2.2       | IDFT . . . . .                                                 | 6         |
| 2.3       | Filtro . . . . .                                               | 7         |
| 2.4       | Misura prestazioni . . . . .                                   | 7         |
| 2.5       | Risultati . . . . .                                            | 8         |
| <b>3</b>  | <b>Prima Implementazione Parallela Naïve</b>                   | <b>9</b>  |
| 3.1       | Misura prestazioni . . . . .                                   | 9         |
| 3.2       | Risultati . . . . .                                            | 10        |
| <b>4</b>  | <b>Ottimizzazioni aritmetiche</b>                              | <b>11</b> |
| 4.1       | Risultati . . . . .                                            | 13        |
| <b>5</b>  | <b>Dimensionamento blocchi</b>                                 | <b>14</b> |
| <b>6</b>  | <b>Shared Memory</b>                                           | <b>17</b> |
| 6.1       | Risultati . . . . .                                            | 18        |
| <b>7</b>  | <b>Loop unrolling</b>                                          | <b>19</b> |
| 7.1       | Risultati . . . . .                                            | 20        |
| <b>8</b>  | <b>Cache L1</b>                                                | <b>22</b> |
| <b>9</b>  | <b>Conclusioni</b>                                             | <b>22</b> |
| <b>10</b> | <b>Estensione: Video Edge Detection</b>                        | <b>23</b> |
| 10.1      | Intro . . . . .                                                | 23        |
| 10.2      | Implementazione naïve . . . . .                                | 23        |
| 10.3      | Ottimizzazione tramite CUDA Streams e Kernel Fusion . . . . .  | 24        |
| 10.4      | Pipelining Avanzato e Risoluzione del GPU Starvation . . . . . | 26        |
| 10.5      | Risultati . . . . .                                            | 27        |

# 1 Studio del problema

L'obiettivo alla base di questo progetto è realizzare un programma CUDA parallelo per effettuare Edge Detection su immagini. L'approccio adottato consiste nel rappresentare l'immagine nel dominio delle frequenze, applicare un filtro passa-alto per enfatizzare le componenti ad alta frequenza (che corrispondono ai bordi), e infine ricostruire l'immagine nel dominio spaziale.

## 1.1 DFT e IDFT

Lo strumento utilizzato per rappresentare l'immagine nel dominio delle frequenze è la trasformata discreta di Fourier (DFT). In generale permette di trasferire un insieme finito di campioni (nel nostro caso i pixel di un'immagine), dal dominio spaziale o temporale al dominio delle frequenze, rappresentando ogni campione come combinazione di onde sinusoidali con frequenze discrete.

$$F[u, v] = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f[x, y] \cdot e^{-j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

Per tornare nel dominio spaziale dopo aver applicato il filtro invece usiamo l'anti-trasformata discreta di Fourier (IDFT).

$$f[x, y] = \frac{1}{MN} \sum_{u=0}^{M-1} \sum_{v=0}^{N-1} F[u, v] \cdot e^{j2\pi\left(\frac{ux}{M} + \frac{vy}{N}\right)}$$

in cui

- $M$  e  $N$  sono le dimensioni dell'immagine di partenza
- $f[x, y]$  è il valore che il pixel dell'immagine assume nel punto di coordinate  $(x, y)$
- $F[u, v]$  è il valore della trasformata nel punto di coordinate  $(u, v)$

## 1.2 Filtro

Dopo aver eseguito la trasformata è necessario applicare un filtro per conservare solo le frequenze necessarie per visualizzare i bordi dell'immagine, ovvero le alte frequenze.

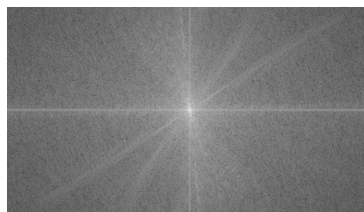


Figura 1: Immagine dopo l'applicazione della trasformata di Fourier

Per comprendere l'azione del filtro, è fondamentale distinguere la rappresentazione visiva dello spettro da quella computazionale. Visivamente, per rendere lo spettro leggibile all'occhio umano, si applica un'operazione di shift dei quadranti (come mostrato in Figura 1): in questa configurazione, le basse frequenze (caratterizzate da un'elevata magnitudo

e quindi da pixel molto chiari) si concentrano al centro dell'immagine, mentre le alte frequenze si sviluppano verso i bordi esterni. Tuttavia, per evitare l'overhead computazionale dovuto al riarrangiamento dei dati in memoria, il kernel CUDA opera direttamente sullo spettro non shiftato. In questa matrice matematica grezza, la disposizione è invertita: le basse frequenze si trovano nei quattro angoli (vertici), mentre le alte frequenze occupano la zona centrale. Il filtro passa-alto implementato agisce quindi calcolando la distanza di ciascun pixel dall'angolo più vicino; se questa distanza è inferiore a un determinato raggio di cutoff, i canali reale e immaginario del pixel vengono azzerati. Questa operazione di filtraggio si rivela particolarmente efficiente su architetture GPU. La condizione di taglio (se azzerare o mantenere un coefficiente) dipende esclusivamente dalle coordinate spaziali  $(x, y)$  del pixel all'interno della matrice e non richiede alcuna informazione sui pixel vicini. Questa natura element-wise, priva di dipendenze logiche o di memoria tra i thread, rende l'algoritmo perfettamente parallelizzabile, consentendo di sfruttare al massimo l'elevato throughput computazionale della GPU.

### 1.3 Formato immagine

Il formato di immagini utilizzato è PGM (Portable GrayMap). Questo formato è molto conveniente per una serie di motivazioni computazionali.

L'immagine infatti è rappresentata in scala di grigi ed ogni pixel è identificato come un singolo valore numerico compreso tra 0 e 255, quindi l'immagine è una singola matrice 2D ( $f[x, y]$ ) a cui possiamo applicare direttamente DFT.

In RGB dovremmo applicare la trasformata a 3 matrici separate per i 3 canali di colori ( $f_R[x, y]$ ,  $f_G[x, y]$ ,  $f_B[x, y]$ ), aumentando la complessità dei calcoli.

Un altro vantaggio offerto da questo formato, è l'assenza di compressione. I valori dei pixel quindi vengono memorizzati senza perdita di dati, fondamentale per il nostro scopo. Utilizzare PGM ci permette quindi di concentrarci maggiormente sull'efficienza e sulla parallelizzazione.

## 2 Implementazione Sequenziale Naïve

Abbiamo realizzato una prima versione del programma: prog.c. Questa versione è scritta in C ed è completamente sequenziale. Realizzare questa versione ci serve per avere una baseline e poterla confrontare con le versioni successive per confrontare lo speedup.

### 2.1 DFT

```
void trasformata(unsigned char *in, MyComplex *out, int width, int height) {  
  
    MyComplex sum;  
    double angle;  
    double pixel;  
  
    for (int y = 0; y < height; y++) {  
        for (int x = 0; x < width; x++) {  
  
            sum.real = 0;  
            sum.imag = 0;  
  
            for (int v = 0; v < height; v++) {  
                for (int u = 0; u < width; u++) {  
  
                    angle = -2.0f * PI * ((double)x * u / width + (double)y * v / height);  
                    pixel = in[v * width + u];  
  
                    sum.real += pixel * cos(angle);  
                    sum.imag += pixel * sin(angle);  
  
                }  
            }  
  
            out[y * width + x] = sum;  
  
        }  
    }  
}
```

Figura 2: Trasformata

La funzione *trasformata* (Figura 2) prende in ingresso:

- **in**: un vettore di *char* che rappresenta i pixel che compongono l'immagine
- **out**: un vettore di *MyComplex* in cui immagazzinare parte reale e immaginaria di una serie di numeri complessi
- **width**: la larghezza dell'immagine
- **height**: l'altezza dell'immagine

*MyComplex* è una struct formata come segue:

```
typedef struct {  
    double real;  
    double imag;  
} MyComplex;
```

Figura 3: MyComplex

*trasformata* è l'implementazione sequenziale della formula

$$F[u, v] = \sum_{x=0}^{M-1} \sum_{y=0}^{N-1} f[x, y] \cdot e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}$$

in cui abbiamo cambiato la rappresentazione dei numeri complessi in linea con il formato di *MyComplex*, che rappresenta tramite due double la parte reale e la parte immaginaria di un numero complesso come mostrato in Figura 3. Avremo:

$$e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})} = e^{j\alpha} = \cos(\alpha) + j\sin(\alpha) = a + jb$$

$$f[x, y] = c + jd$$

di conseguenza

$$f[x, y] \cdot e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})} = (a + jb) \cdot (c + jd) = ac + jad + jbc - bd$$

considerando che  $f[x, y]$  è un valore reale,  $d=0$  e quindi

$$f[x, y] \cdot e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})} = ac + jbc$$

dato che  $a = \cos(\alpha)$  e  $b = \sin(\alpha)$

$$\text{Re}\{f[x, y] \cdot e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}\} = c \cdot \cos(\alpha)$$

$$\text{Imm}\{f[x, y] \cdot e^{-j2\pi(\frac{ux}{M} + \frac{vy}{N})}\} = c \cdot \sin(\alpha)$$

in cui  $c$  corrisponde al valore di ciascun pixel dell'immagine di partenza (*pixel* in Figura 2).

## 2.2 IDFT

```
void anti_trasformata(MyComplex *in, unsigned char *out, int width, int height) {
    double sum;
    double angle;
    MyComplex trasf;

    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {
            sum = 0;

            for (int v = 0; v < height; v++) {
                for (int u = 0; u < width; u++) {
                    angle = 2.0f * PI * ((double)x * u / width + (double)y * v / height);

                    trasf = in[v * width + u];
                    sum += trasf.real * cos(angle) - trasf.imag * sin(angle);
                }
            }

            sum /= (width * height);
            out[y * width + x] = (unsigned char)(sum < 0 ? 0 : (sum > 255 ? 255 : sum));
        }
    }
}
```

Figura 4: Anti-trasformata

La funzione *anti\_trasformata* (Figura 4) prende in ingresso:

- **in**: un vettore di *MyComplex* che contiene i valori della trasformata
- **out**: un vettore di *char* in cui salvare i valori da dare ai pixel dell'immagine
- **width**: la larghezza dell'immagine
- **height**: l'altezza dell'immagine

Come già visto per la trasformata, anche per l'anti-trasformata applichiamo le stesse semplificazioni:

$$e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})} = e^{j\alpha} = \cos(\alpha) + j\sin(\alpha) = a + jb$$

$$F[u, v] = c + jd$$

di conseguenza:

$$F[u, v] \cdot e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})} = (a + jb) \cdot (c + jd) = ac + jad + jbc - bd = ac - bd + j(ad + bc)$$

dato che  $a = \cos(\alpha)$  e  $b = \sin(\alpha)$

$$\text{Re}\{F[u, v] \cdot e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}\} = c \cdot \cos(\alpha) - d \cdot \sin(\alpha)$$

$$\text{Imm}\{F[u, v] \cdot e^{j2\pi(\frac{ux}{M} + \frac{vy}{N})}\} = c \cdot \sin(\alpha) + d \cdot \cos(\alpha)$$

in cui  $c$  e  $d$  corrispondono al valore reale e immaginario della trasformata in ogni pixel dell'immagine. La funzione deve resistere i valori dei pixel dell'immagine, per questo siamo interessati solo alla parte reale.

## 2.3 Filtro

```
void filtro(MyComplex *dft, int width, int height, float cutoff) {

    int Xmezzi = width / 2;
    int Ymezzi = height / 2;
    float d;

    for (int y = 0; y < height; y++) {
        for (int x = 0; x < width; x++) {

            float dx = (x < width / 2) ? x : (width - x);
            float dy = (y < height / 2) ? y : (height - y);
            float d = sqrtf(dx * dx + dy * dy);

            // EDGE DETECTION (Filtro Passa-Alto)
            if (d < cutoff) {
                dft[y * width + x].real = 0.0f;
                dft[y * width + x].imag = 0.0f;
            }
        }
    }
}
```

Figura 5: Filtro sequenziale

La funzione *filtro* prende in ingresso i valori della trasformata (*MyComplex \*dft*) e applica un filtro all'immagine in base al valore passato tramite (*float raggio*). Come abbiamo spiegato in 1.2, vogliamo azzerare tutte le frequenze che si trovano in prossimità degli angoli (portando a 0 i pixel con distanza *d* dall'angolo minore di *raggio*) perchè non sono bordi. Il *raggio* che useremo è 30 su un'immagine di 288x512 pixel.

## 2.4 Misura prestazioni

```
start = clock();

trasformata(img.data, dft, img.width, img.height);

end = clock();
tempo = ((double)(end - start)) / CLOCKS_PER_SEC ;
printf("Tempo trasformata: %f\n", tempo);
```

Figura 6: Calcolo prestazioni codice sequenziale

Per misurare il tempo di esecuzione (in questo caso per *trasformata*) utilizziamo la libreria **time.h** di C. In modo analogo per *filtro* e *anti\_trasformata*.

## 2.5 Risultati

Il codice sequenziale è stato eseguito su un Apple MacBook con chip M3 (Apple Silicon, ARM64), 16 GB di RAM, macOS, utilizzando un'immagine da  $512 \times 288$  pixel, ottenendo i seguenti risultati:

| Funzione                | Tempo    |
|-------------------------|----------|
| <i>trasformata</i>      | 205.07 s |
| <i>filtro</i>           | 0.412 ms |
| <i>anti_trasformata</i> | 204.23 s |
| <b>Totale</b>           | 409.30 s |

Tabella 1: Tempi di esecuzione **DFTSeq.c**



Figura 7: Immagine originale

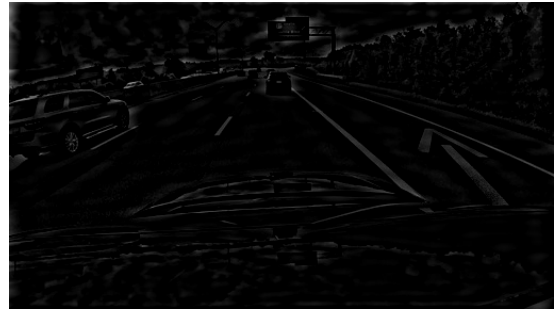


Figura 8: Immagine filtrata

### 3 Prima Implementazione Parallela Naïve

Nella prima implementazione parallela su GPU è stato assegnato a ogni thread un singolo pixel dell'immagine di partenza, a cui è stata applicata la trasformata, il filtro e l'anti-trasformata.

```
__global__ void dft2D(unsigned char *in, MyComplex *out, int width, int height) {

    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    MyComplex sum;
    sum.real = 0;
    sum.imag = 0;

    double angle;
    double pixel;

    if (x < width && y < height) {

        for (int v = 0; v < height; v++) {
            for (int u = 0; u < width; u++) {

                angle = -2.0f * PI * ((double)x * u / width + (double)y * v / height);
                pixel = in[v * width + u];
                sum.real += pixel * cos(angle);
                sum.imag += pixel * sin(angle);

            }
        }
        out[y * width + x] = sum;
    }
}
```

Figura 9: Trasformata nella prima versione parallela su Cuda

In modo analogo è stato fatto per la funzione *filtro* e *idft*.

#### 3.1 Misura prestazioni

Per misurare le prestazioni da qui in avanti utilizzeremo *Nsight Compute* che ci permette di fare un'analisi approfondita e utilizzare questa prima versione (**DFTcudanaive.cu**) come base per il confronto con le versioni parallelizzate successive e il calcolo dello speedup.

## 3.2 Risultati

Mostriamo nella tabella sottostante il tempo di calcolo e lo speedup rispetto alla versione sequenziale:

| Funzione      | Tempo        | Speedup      |
|---------------|--------------|--------------|
| <i>dft</i>    | 27.50 s      | <b>7.39</b>  |
| <i>filtro</i> | 6.50 $\mu$ s | <b>63.38</b> |
| <i>idft</i>   | 27.47 s      | <b>7.41</b>  |
| <b>Totale</b> | 54.97 s      | <b>7.40</b>  |

Tabella 2: Tempi di esecuzione **DFTcudanaive.cu**

Analizziamo ora con *Nsight* il kernel di **DFT** e **IDFT**, dato che la funzione *filtro* non determina in maniera significativa il tempo di calcolo totale:

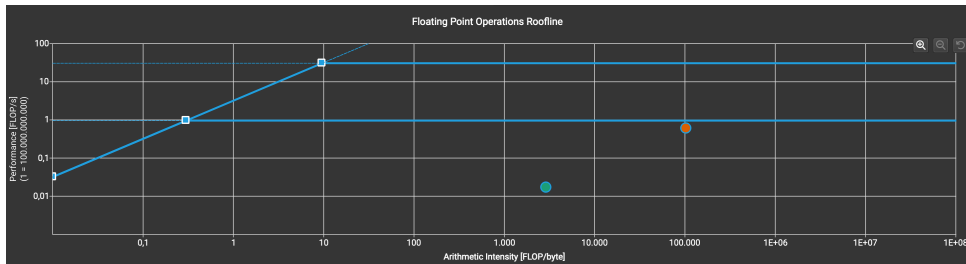


Figura 10: Roofline Model DFT Naive

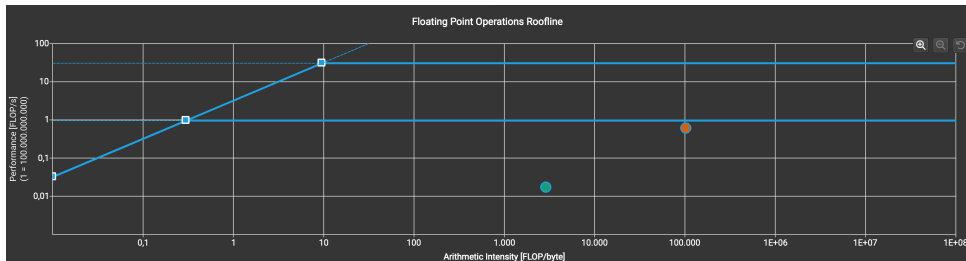


Figura 11: Roofline Model IDFT Naive

I due grafici Roofline mettono in relazione la capacità computazionale del dispositivo con l'intensità aritmetica del kernel, permettendo di analizzare le prestazioni di **DFTcudanaive.cu**.

Le due rette orizzontali rappresentano i limiti teorici di calcolo della GPU: quella inferiore corrisponde alla doppia precisione, mentre quella superiore alla singola precisione. La retta inclinata indica invece il limite imposto dalla bandwidth di memoria.

I punti arancioni rappresentano le prestazioni raggiunte dalle operazioni in doppia precisione, mentre i punti verdi quelle relative alla singola precisione.

I risultati mostrano un'elevata intensità aritmetica, coerente con la natura computazionalmente onerosa della Trasformata di Fourier. I kernel possono quindi essere classificati

come *compute-bound*, cioè limitati principalmente dalla capacità di calcolo della GPU e non dalla bandwidth di memoria, come confermato dalla distanza dei punti dalla retta inclinata.

Un ulteriore aspetto importante riguarda la doppia precisione: il punto arancione risulta molto vicino al corrispondente limite teorico, indicando un ridotto margine di miglioramento senza utilizzare hardware più performante. Al contrario, la singola precisione offre un ampio margine di miglioramento.

Osserviamo ulteriori grafici che riguardano l'utilizzo delle varie pipeline:

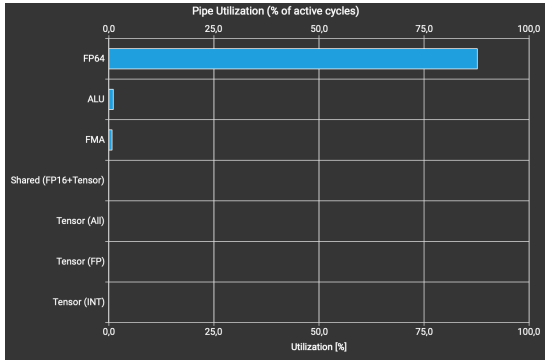


Figura 12: Pipeline utilization DFT

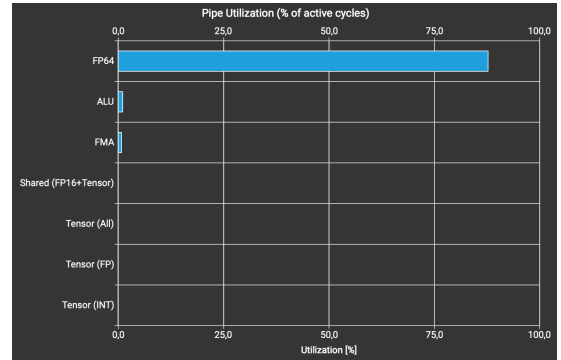


Figura 13: Pipeline utilization IDFT

Si osserva un evidente squilibrio nell'utilizzo delle risorse dedicate alle operazioni in doppia precisione.

I dati raccolti suggeriscono che un passaggio da *double* a *float* potrebbe determinare un significativo incremento delle prestazioni a discapito di una minima perdita di precisione nei calcoli. Un ulteriore margine di miglioramento riguarda la componente in singola precisione, che presenta ancora possibilità di ottimizzazione.

## 4 Ottimizzazioni aritmetiche

Osservando il codice della prima versione parallela in cuda e i relativi dati raccolti, abbiamo apportato le seguenti modifiche:

- **Passaggio da *double* a *float*:** Nella prima versione di codice cuda parallelo e nella versione sequenziale il tipo di dato usato era *double*. Nonostante offra una maggiore precisione, è molto impattante sulle prestazioni. Per questo è stato scelto di utilizzare variabili di tipo *float*, che offrono comunque una buona precisione e sono computazionalmente meno costose.
- **FMA (Fused Multiply-Add):** La trasformata di Fourier deve eseguire operazioni del tipo  $(a * b + c)$ . Utilizzare *FMA* per implementare queste operazioni in un'unica istruzione permette di risparmiare tempo di calcolo.
- **sin e cos:** Per i kernel di *dft* e *idft* abbiamo bisogno di calcolare ripetutamente i valori di seno e coseno. Utilizzando l'istruzione *sincosf* calcoliamo in una singola

istruzione il valore del seno e del coseno di un determinato angolo migliorando l'efficienza.

- **-use\_fast\_math:** Usato in fase di compilazione permette di riconoscere operazioni adatte a *FMA* e sostituire *sincosf* con un'operazione di basso livello hardware *\_\_sincosf* che è più veloce.
- **Precalcolo fattore angolare orizzontale e verticale:** In **DFTcudanaive.cu**  $\alpha$  veniva ricalcolato per ogni pixel dell'immagine di partenza. Per ridurre il numero di operazioni svolte nei kernel, si precalcolano il fattore angolare orizzontale e verticale prima di entrare nel loop principale. Successivamente, durante l'iterazione, tali valori vengono moltiplicati rispettivamente per  $u$  e  $v$ , evitando divisioni e moltiplicazioni ridondanti ad ogni passo. In questo modo il calcolo dell'angolo risulta meno costoso e il kernel beneficia di una migliore efficienza computazionale.

Tutte le modifiche apportate si trovano nel file **DFTcudaopt1.cu**. In particolare mostriamo ora il kernel *dft\_opt*, in maniera analoga è stato ottimizzato *idft*:

```
__global__ void dft_opt1(unsigned char *in, MyComplex *out, int width, int height) {

    int x = blockIdx.x * blockDim.x + threadIdx.x;
    int y = blockIdx.y * blockDim.y + threadIdx.y;

    if (x >= width || y >= height) return;

    float sumReal = 0.0f;
    float sumImag = 0.0f;

    // Precalcola i fattori angolari per questo thread
    float angleX = -2.0f * PI * (float)x / width;
    float angleY = -2.0f * PI * (float)y / height;

    for (int v = 0; v < height; v++) {

        // sin/cos della componente verticale -> costanti per tutto il loop su u
        float cosV, sinV;
        sincosf(angleY * v, &sinV, &cosV);

        for (int u = 0; u < width; u++) {

            // sin/cos della componente orizzontale -> costanti su tutto il loop su v
            float cosU, sinU;
            sincosf(angleX * u, &sinU, &cosU);

            // cos(U+V) = cosU*cosV - sinU*sinV
            float c = __fmaf_rn(cosU, cosV, -sinU * sinV);
            // sin(U+V) = sinU*cosV + cosU*sinV
            float s = __fmaf_rn(sinU, cosV, cosU * sinV);

            float pixel = (float)in[v * width + u];

            // sum = pixel*c + sumReal
            sumReal = __fmaf_rn(pixel, c, sumReal);
            sumImag = __fmaf_rn(pixel, s, sumImag);

        }

    }

    out[y * width + x] = { sumReal, sumImag };
}
```

Figura 14: Trasformata dopo aver applicato le ottimizzazioni matematiche

## 4.1 Risultati

Mostriamo nella tabella sottostante il tempo di calcolo e lo speedup rispetto alla versione sequenziale e alla prima versione parallela:

| Funzione         | Tempo        | Speedup (sequenziale) | Speedup (cuda_naive) |
|------------------|--------------|-----------------------|----------------------|
| <i>dft_opt1</i>  | 243.93 ms    | <b>833.11</b>         | <b>112.74</b>        |
| <i>filtro</i>    | 5.82 $\mu$ s | <b>51.20</b>          | <b>1.10</b>          |
| <i>idft_opt1</i> | 185.43 ms    | <b>1097.61</b>        | <b>148.14</b>        |
| <b>Totale</b>    | 429.36 ms    | <b>947.34</b>         | <b>128.03</b>        |

Tabella 3: Tempi di esecuzione **DFTcudaOpt1.cu**

Nelle immagini sottostanti si può chiaramente notare che passare da *double* a *float* e le varie ottimizzazioni matematiche non hanno portato a una perdita di precisione o diminuzione della qualità di immagine.

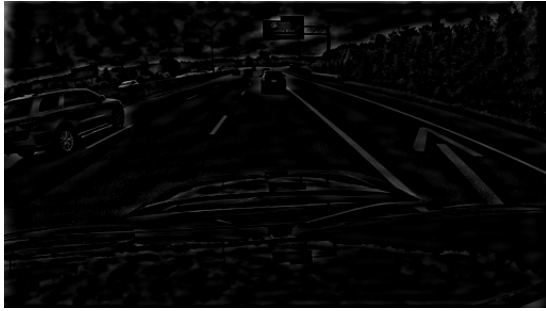


Figura 15: DFTseq.c

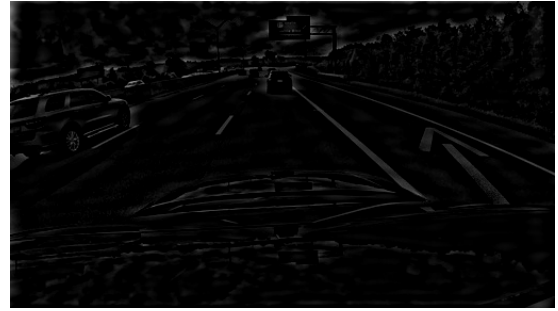


Figura 16: DFTcudaOpt1.cu

Osservando i roofline (Figura 17 e 18) si può notare che i punti verdi che rappresentano i calcoli in singola precisione sono molto più vicini al loro limite teorico rispetto alla versione **DFTcudanaive.cu**.

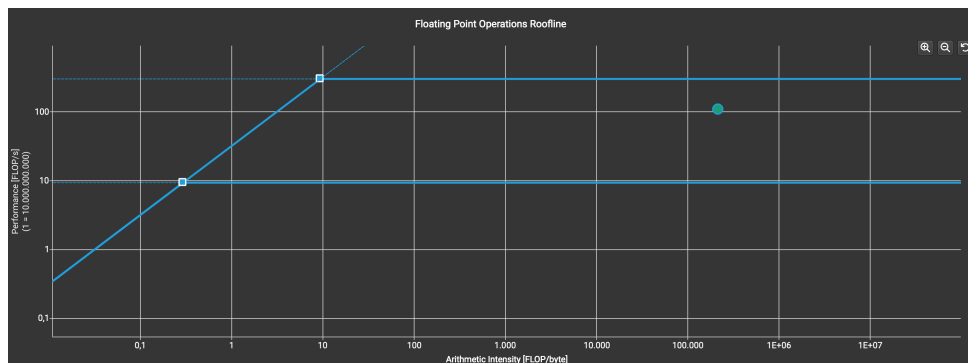


Figura 17: Roofline Model DFT Opt1

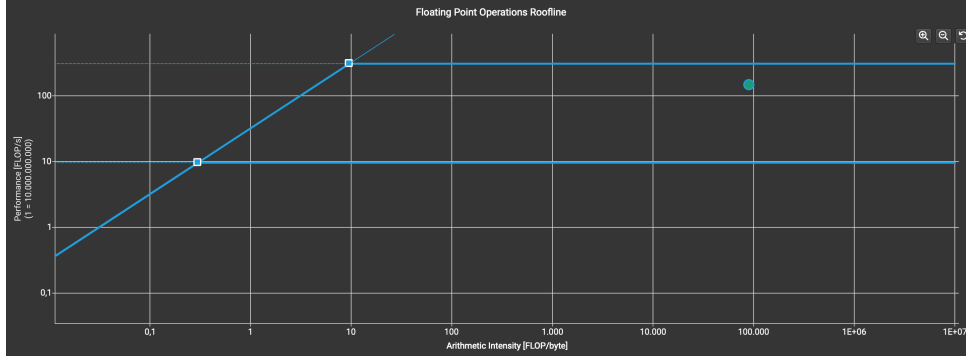


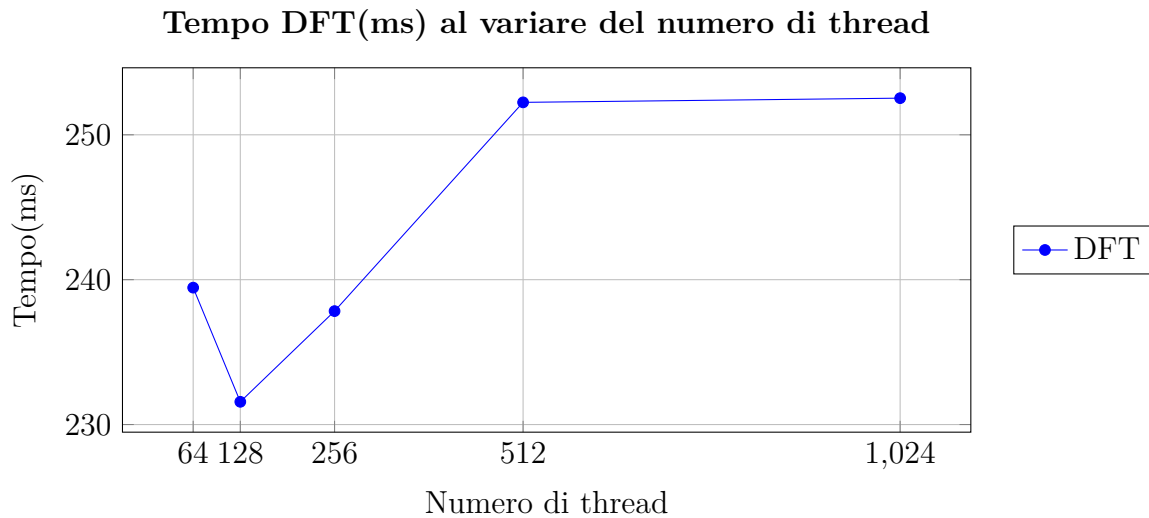
Figura 18: Roofline Model IDFT Opt1

## 5 Dimensionamento blocchi

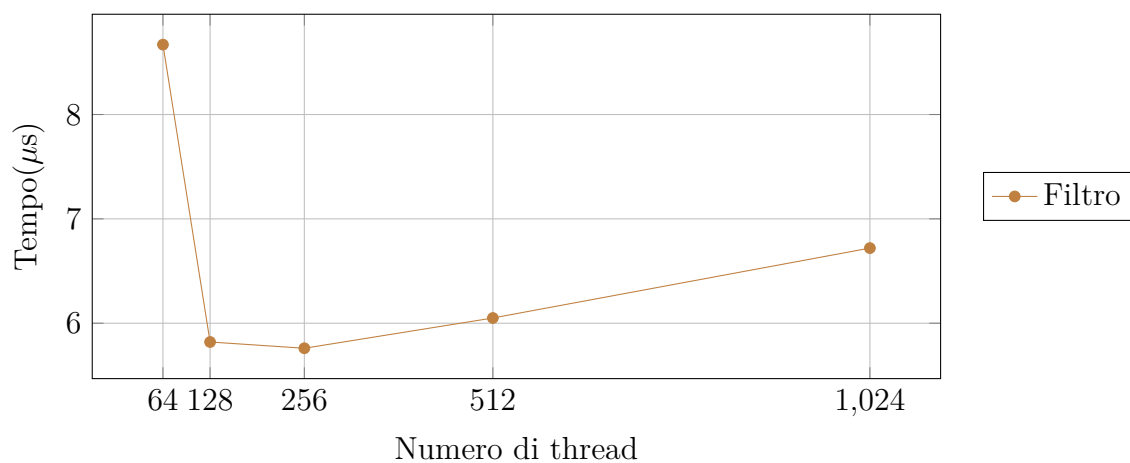
Un'altra tecnica per migliorare ulteriormente le prestazioni è dimensionare correttamente i blocchi, per distribuire uniformemente il carico di lavoro in ogni kernel. Abbiamo effettuato diverse prove, modificando **DFTcudaOpt1.cu**. I risultati raccolti sono mostrati in seguito:

|                             | 64     | 128    | 256    | 512    | 1024   |
|-----------------------------|--------|--------|--------|--------|--------|
| Tempo DFT (ms)              | 239.45 | 231.57 | 237.83 | 252.24 | 252.53 |
| Tempo filtro ( $\mu$ s)     | 8.67   | 5.82   | 5.76   | 6.05   | 6.72   |
| Tempo IDFT (ms)             | 185.32 | 179.39 | 181.07 | 190.04 | 190.20 |
| Occupancy DFT (%)           | 88.11  | 89.44  | 90.07  | 92.32  | 98.23  |
| Occupancy filtro (%)        | 31.33  | 58.97  | 66.93  | 74.28  | 83.59  |
| Occupancy IDFT (%)          | 87.87  | 90.04  | 90.94  | 93.01  | 98.27  |
| Speedup DFT (cuda_naive)    | 114.84 | 118.75 | 115.63 | 109.02 | 108.90 |
| Speedup filtro (cuda_naive) | 0.75   | 1.12   | 1.13   | 1.07   | 0.97   |
| Speedup IDFT (cuda_naive)   | 148.23 | 153.13 | 151.71 | 144.55 | 144.43 |

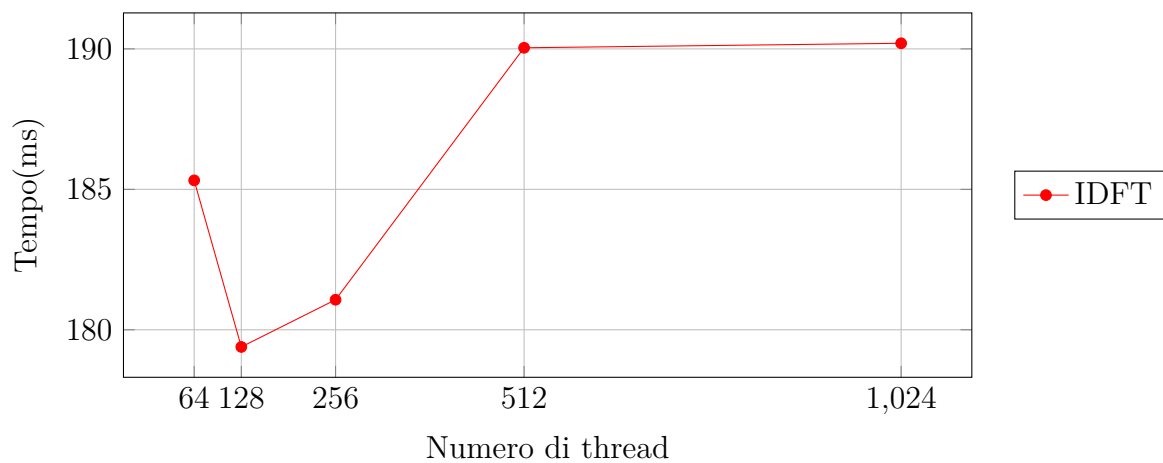
Tabella 4: Prestazioni dei kernel e speedup cambiando il numero di thread



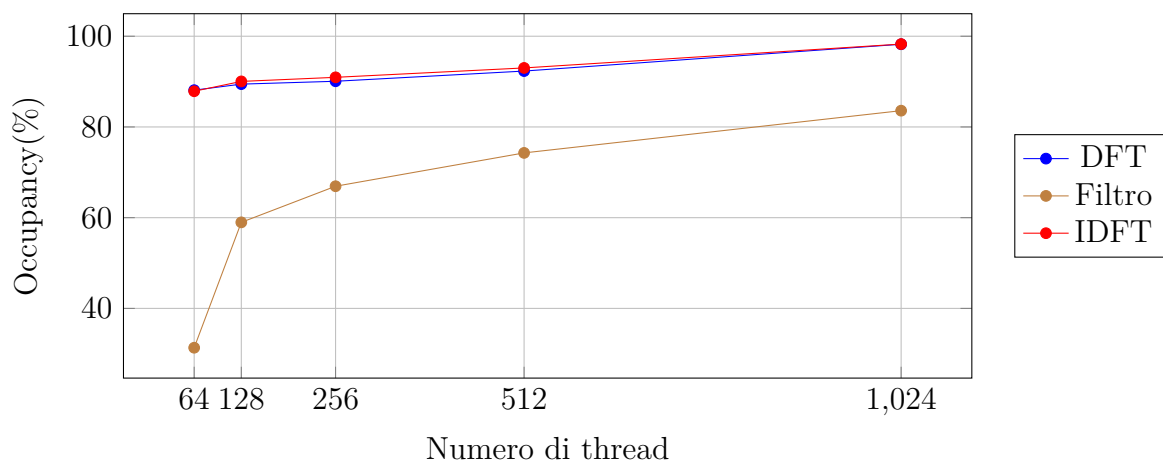
Tempo Filtro( $\mu s$ ) al variare del numero di thread

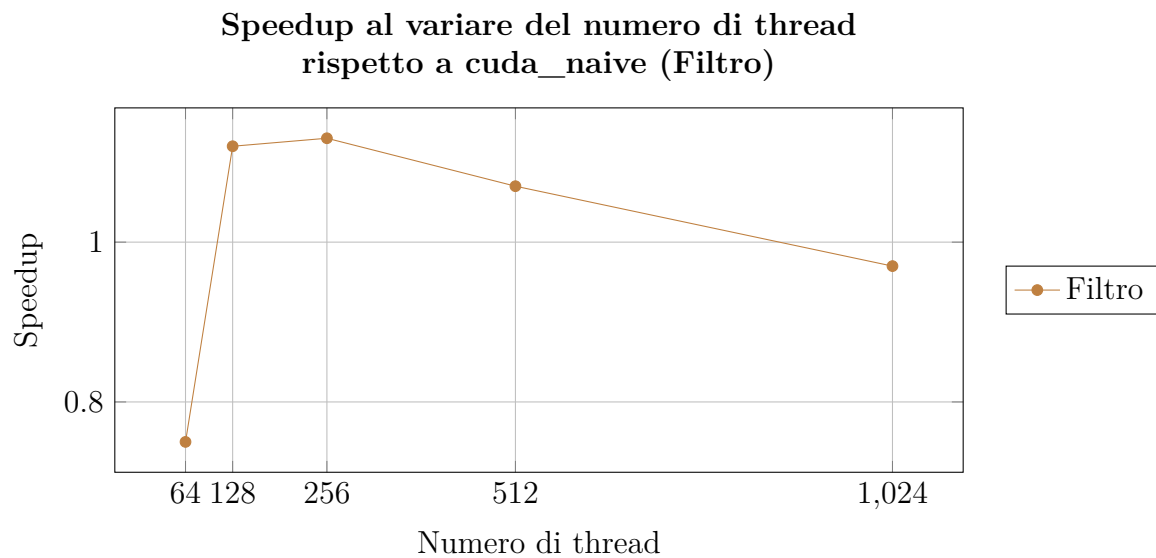
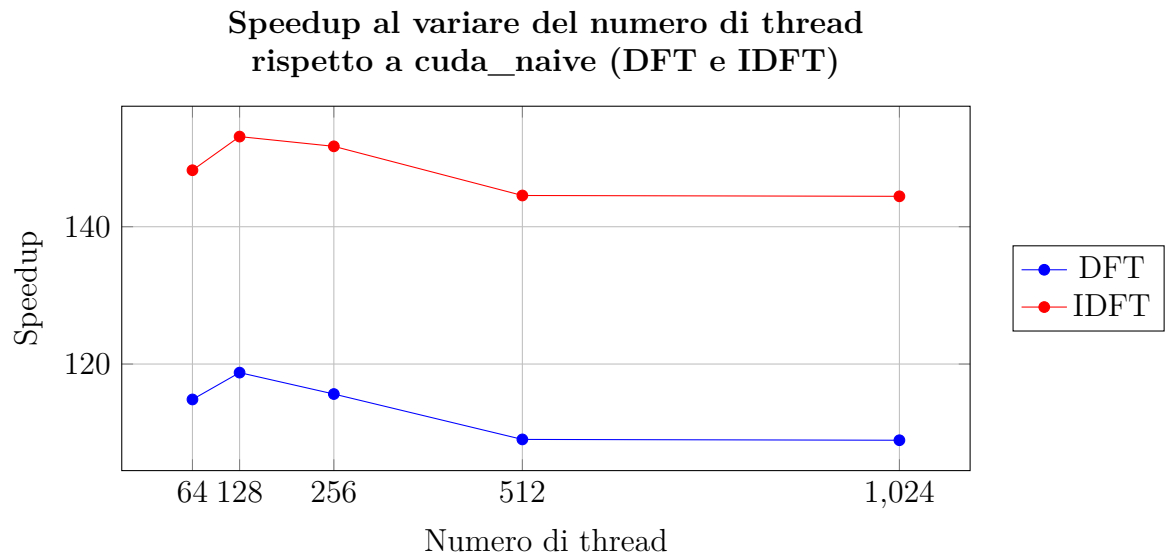


Tempo IDFT(ms) al variare del numero di thread



Occupancy al variare del numero di thread





Alla luce dei risultati ottenuti, possiamo affermare che la configurazione ottimale è 128(16x8) per i kernel di DFT e IDFT e 256(16x16) per il Filtro. Queste configurazioni verranno mantenute nelle ottimizzazioni successive.

## 6 Shared Memory

Nei kernel di **DFT** e **IDFT**, i thread accedono ai dati delle matrici di input in modo sequenziale, e gli stessi dati vengono riutilizzati da più thread. Questo rende possibile sfruttare la *shared memory* per favorire la collaborazione tra i thread all'interno di ciascun blocco.

Nella nuova ottimizzazione (**DFTcudaOpt2.cu**), ciascun thread legge un elemento specifico dalla *global memory* e lo copia in un array collocato in *shared memory*. Dopo una sincronizzazione tra i thread, i dati vengono riutilizzati direttamente dalla *shared memory* per eseguire i calcoli, riducendo così il numero di accessi alla memoria globale. Una volta completata l'elaborazione del blocco corrente, i thread ripetono il trasferimento caricando una nuova porzione di dati dalla memoria globale alla *shared memory*.

Questo approccio consente una collaborazione efficiente tra i thread, poiché ogni elemento dell'input viene caricato una sola volta per blocco e poi riutilizzato. Tuttavia, presenta anche alcune limitazioni che possono ridurre il guadagno in termini di prestazioni. In particolare, è necessario sincronizzare i thread dopo ogni fase di caricamento in memoria condivisa (`__syncthreads()`), introducendo un overhead che può incidere sullo speedup complessivo.

Per quanto riguarda il kernel del filtro, invece, l'utilizzo della *shared memory* non risulta vantaggioso, poiché non vi è un riutilizzo significativo dei dati tra i thread: ogni thread opera su dati distinti, rendendo trascurabile il beneficio derivante dalla loro condivisione.

L'array in memoria condivisa è stato definito come:

```
__shared__ float tile[TILE_H][TILE_W];
```

Figura 19: dft\_shared

```
__shared__ float tileReal[TILE_H][TILE_W];  
__shared__ float tileImag[TILE_H][TILE_W];
```

Figura 20: idft\_shared

in cui **TILE\_H** e **TILE\_W** corrispondono rispettivamente a 16 e 8 ovvero le dimensioni dei blocchi dei kernel DFT e IDFT. Per *idft\_shared* usiamo due array in quanto (coerentemente con il formato di **MyComplex** definito in precedenza) abbiamo una parte reale e una immaginaria.

## 6.1 Risultati

Come già discusso in precedenza, il problema in esame presenta una natura **compute-bound**. Inoltre, la necessità di sincronizzazione tra i thread può introdurre ulteriori rallentamenti. Di conseguenza, ci si aspetta che l'utilizzo della shared memory non produca un miglioramento significativo delle prestazioni.

Mostriamo nella tabella successiva il tempo di calcolo e lo speedup di **DFTcudaOpt2.cu** rispetto alla prima ottimizzazione (**DFTcudaOpt1.cu**):

| Funzione           | Tempo        | Speedup (opt1) |
|--------------------|--------------|----------------|
| <i>dft_shared</i>  | 189.80 ms    | <b>1.29</b>    |
| <i>filtro</i>      | 5.82 $\mu$ s | <b>1</b>       |
| <i>idft_shared</i> | 190.14 ms    | <b>0.98</b>    |
| <b>Totale</b>      | 379.94 ms    | <b>1</b>       |

Tabella 5: Tempi di esecuzione **DFTcudaOpt2.cu**

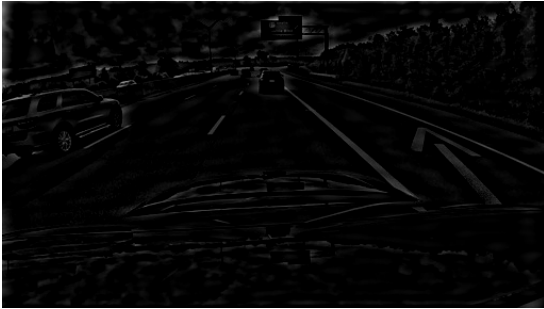


Figura 21: DFTcudaOpt1.cu

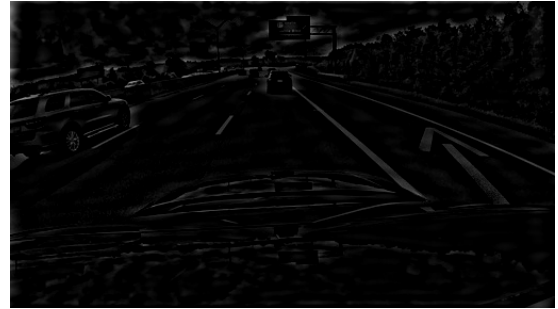


Figura 22: DFTcudaOpt2.cu

I risultati raccolti rispecchiano ciò che ci aspettavamo. In particolare nella Tabella 5 possiamo notare che lo speedup della *dft\_shared* rispetto alla precedente versione ottimizzata è basso e per la *idft\_shared* causa addirittura un leggero slowdown. Passiamo a una nuova versione (**DFTcudaOpt2 \_\_1.cu**) in cui manteniamo la shared memory solo per il kernel della DFT. I risultati raccolti sono mostrati nelle tabelle successive:

| Funzione          | Tempo        | Speedup (sequenziale) | Speedup (cuda_naive) |
|-------------------|--------------|-----------------------|----------------------|
| <i>dft_shared</i> | 189.80 ms    | <b>1070.71</b>        | <b>144.89</b>        |
| <i>filtro</i>     | 5.82 $\mu$ s | <b>51.20</b>          | <b>1.12</b>          |
| <i>idft_opt1</i>  | 183.96 ms    | <b>1106.38</b>        | <b>149.33</b>        |
| <b>Totale</b>     | 373.76 ms    | <b>1088.27</b>        | <b>147.07</b>        |

Tabella 6: Tempi di esecuzione **DFTcudaOpt2 \_\_1.cu**

Nella tabella 6 sono raccolti i tempi di esecuzione dei kernel di **DFTcudaOpt2 \_\_1.cu** e lo speedup rispetto alla versione sequenziale e alla prima versione parallela.

| Funzione          | Tempo        | Speedup (opt1) | Speedup (opt2) |
|-------------------|--------------|----------------|----------------|
| <i>dft_shared</i> | 189.80 ms    | <b>1.29</b>    | <b>1</b>       |
| <i>filtro</i>     | 5.82 $\mu$ s | <b>1</b>       | <b>1</b>       |
| <i>idft_opt1</i>  | 183.96 ms    | <b>1.01</b>    | <b>1.03</b>    |
| <b>Totale</b>     | 373.76 ms    | <b>1.15</b>    | <b>1.02</b>    |

Tabella 7: Tempi di esecuzione **DFTcudaOpt2\_1.cu**

Nella tabella 7 mostriamo il tempo di esecuzione di **DFTcudaOpt2\_1.cu** e lo speedup rispetto a **DFTcudaOpt1.cu** e **DFTcudaOpt2.cu**. La versione definitiva e le versioni successive avranno quindi *dft\_shared* e *idft\_opt1* con blocchi di dimensione 16x8. Per il filtro manteniamo la dimensione del blocco 16x16.



Figura 23: DFTcudaOpt2.cu



Figura 24: DFTcudaOpt2\_1.cu

## 7 Loop unrolling

Per incrementare ulteriormente le prestazioni abbiamo fatto unrolling del ciclo più interno di **DFT** e **IDFT**. Abbiamo testato diversi fattori di unrolling (4,8,16) per valutare quale configurazione offrisse il miglior compromesso in termini di prestazioni. Di seguito mostriamo come è stato realizzato unroll 4, in modo analogo è stato fatto per 8 e 16.

```
#pragma unroll 4
for (int du = 0; du < TILE_W; du++) {

    float cosU, sinU;
    sincosf(angleX * (tileU + du), &sinU, &cosU);

    // Formula addizione con FMA
    float c = __fmaf_rn(cosU, cosV, -sinU * sinV);
    float s = __fmaf_rn(sinU, cosV, cosU * sinV);

    float pixel = tile[dv][du];

    // Accumulazione con FMA
    sumReal = __fmaf_rn(pixel, c, sumReal);
    sumImag = __fmaf_rn(pixel, s, sumImag);
}
```

Figura 25: Unroll 4 per **DFT**

Dato che *TILE\_W* è di dimensione fissa, il loop unrolling viene realizzato in automatico dal compilatore aggiungendo semplicemente **#pragma unroll 4**.

```

for (int u = 0; u < width; u += 4) {
    float cosU0, sinU0, cosU1, sinU1, cosU2, sinU2, cosU3, sinU3;
    sincosf(angleX * (u + 0), &sinU0, &cosU0);
    sincosf(angleX * (u + 1), &sinU1, &cosU1);
    sincosf(angleX * (u + 2), &sinU2, &cosU2);
    sincosf(angleX * (u + 3), &sinU3, &cosU3);

    MyComplex coeff0 = in[v * width + (u + 0)];
    MyComplex coeff1 = in[v * width + (u + 1)];
    MyComplex coeff2 = in[v * width + (u + 2)];
    MyComplex coeff3 = in[v * width + (u + 3)];

    // Step 0
    float c0 = __fmaf_rn(cosU0, cosV, -sinU0 * sinV);
    float s0 = __fmaf_rn(sinU0, cosV, cosU0 * sinV);
    float st1_0 = __fmaf_rn(-coeff0.imag, s0, sum);
    sum = __fmaf_rn(coeff0.real, c0, st1_0);

    // Step 1
    float c1 = __fmaf_rn(cosU1, cosV, -sinU1 * sinV);
    float s1 = __fmaf_rn(sinU1, cosV, cosU1 * sinV);
    float st1_1 = __fmaf_rn(-coeff1.imag, s1, sum);
    sum = __fmaf_rn(coeff1.real, c1, st1_1);

    // Step 2
    float c2 = __fmaf_rn(cosU2, cosV, -sinU2 * sinV);
    float s2 = __fmaf_rn(sinU2, cosV, cosU2 * sinV);
    float st1_2 = __fmaf_rn(-coeff2.imag, s2, sum);
    sum = __fmaf_rn(coeff2.real, c2, st1_2);

    // Step 3
    float c3 = __fmaf_rn(cosU3, cosV, -sinU3 * sinV);
    float s3 = __fmaf_rn(sinU3, cosV, cosU3 * sinV);
    float st1_3 = __fmaf_rn(-coeff3.imag, s3, sum);
    sum = __fmaf_rn(coeff3.real, c3, st1_3);
}

```

Figura 26: Unroll 4 per IDFT

Per la IDFT invece l'unrolling va fatto "manualmente" come mostrato nella Figura 26.

## 7.1 Risultati

I tre livelli di unroll sono raccolti nei file **DFTcudaOpt3\_4.cu**, **DFTcudaOpt3\_8.cu**, **DFTcudaOpt3\_16.cu** e le prestazioni sono analizzate nelle tabelle di seguito:

|                       | 0      | 4      | 8      | 16     |
|-----------------------|--------|--------|--------|--------|
| <b>Tempo DFT(ms)</b>  | 189.80 | 189.56 | 189.44 | 98.87  |
| <b>Tempo IDFT(ms)</b> | 183.96 | 182.89 | 182.96 | 180.27 |

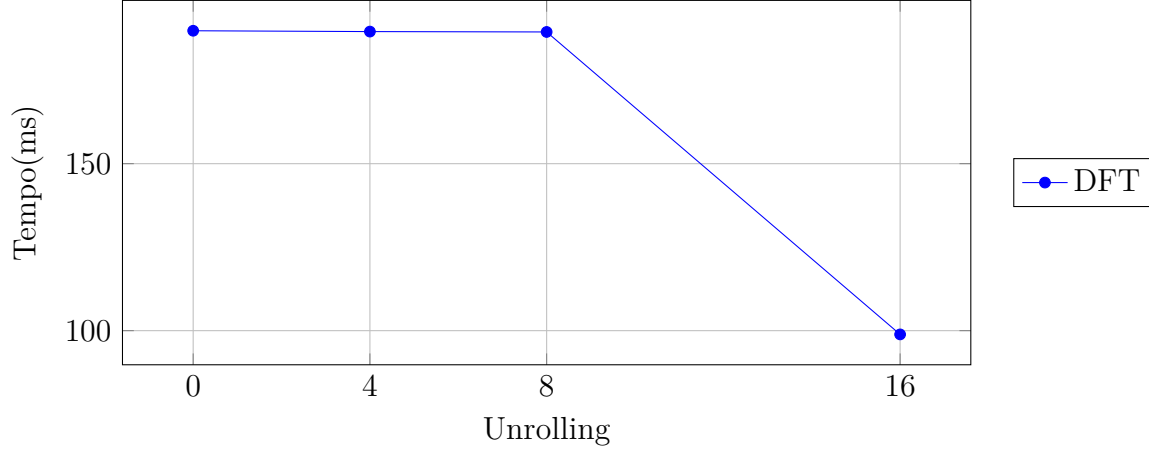
Tabella 8: Prestazioni dei kernel cambiando fattore di unrolling

Notiamo subito che l'unrolling 16 offre vantaggi significativi per la **DFT** mentre la **IDFT** rimane invariata. Rifacciamo unrolling adottando quindi shared memory anche per idft e mostriamo i risultati (file **DFTcudaOpt4\_4.cu**, **DFTcudaOpt4\_8.cu**, **DFTcudaOpt4\_16.cu**):

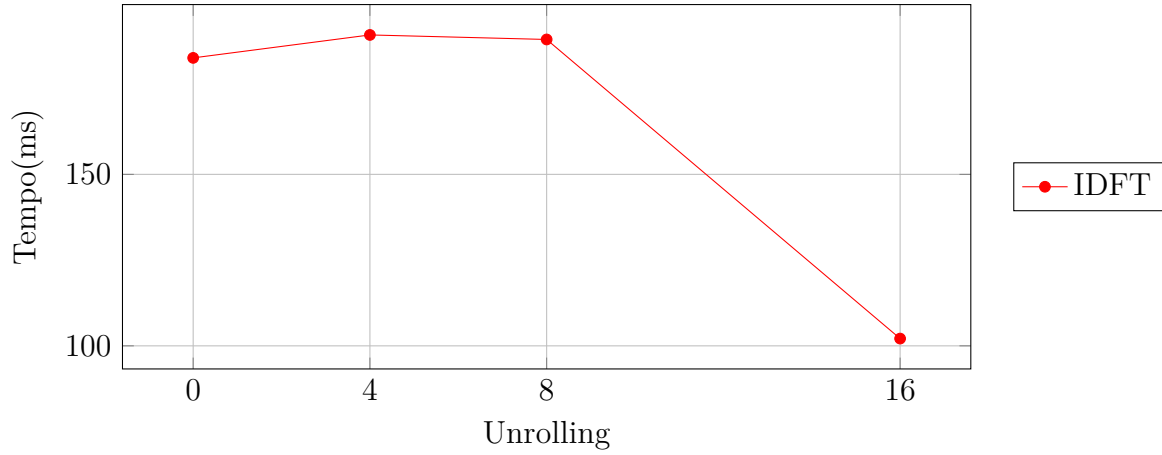
|                       | 0      | 4      | 8      | 16     |
|-----------------------|--------|--------|--------|--------|
| <b>Tempo DFT(ms)</b>  | 189.80 | 189.55 | 189.44 | 98.88  |
| <b>Tempo IDFT(ms)</b> | 183.96 | 190.65 | 189.33 | 102.12 |

Tabella 9: Prestazioni dei kernel cambiando fattore di unrolling

**Tempo DFT(ms) al variare del livello di unrolling**



**Tempo IDFT(ms) al variare del livello di unrolling**



| Funzione           | Tempo     | Speedup(sequenziale) | Speedup(cuda_naive) |
|--------------------|-----------|----------------------|---------------------|
| <i>dft_shared</i>  | 98.88 ms  | <b>2073.93</b>       | <b>278.11</b>       |
| <i>idft_shared</i> | 102.12 ms | <b>1999.90</b>       | <b>268.99</b>       |
| <b>Totale</b>      | 201 ms    | <b>2036.32</b>       | <b>273.48</b>       |

Tabella 10: DFTcudaOpt4\_16.cu

Osservando i grafici e le tabelle possiamo notare che fare unrolling 16 offre dei vantaggi significativi, a patto che sia utilizzato shared memory sia per *dft* che *idft*.

## 8 Cache L1

In questa ottimizzazione abbiamo apportato delle modifiche alla cache L1:

- **`__restrict` sui puntatori nella firma del kernel:** questa istruzione dice al compilatore che in e out non si sovrappongono in memoria
- **coalescenza in memoria:** la scrittura di `MyComplex` (due float) può essere fatta come store vettoriale se la struct è allineata a 8 byte e quindi in una singola transazione invece di due. Accesso Coalescente: l'hardware legge/scrive l'intero blocco in un solo ciclo di clock. È fondamentale per prestazioni ottimali. Se i thread accedessero a locazioni di memoria sparse in modo casuale, il sistema dovrebbe eseguire più transazioni separate, rallentando drasticamente il programma.

Dopo aver analizzato le prestazioni, non è stato registrato alcun miglioramento. Questo è dovuto principalmente al fatto che il nostro problema è **compute-bound**, non **memory-bound**. Per questo motivo gli accessi in memoria sono comunque limitati dai calcoli computazionali. Per eventuali sviluppi futuri abbiamo comunque deciso di mantenere questa ottimizzazione.

## 9 Conclusioni

Siamo partiti da una prima versione sequenziale in c, per avere una baseline da poter confrontare con le versioni successive e analizzare lo speedup. Da questa prima versione siamo passati a una prima versione parallela naive in cuda. Le ottimizzazioni che sono state fatte rispetto alla prima versione naive in cuda sono:

- **Ottimizzazioni aritmetiche:** le prime ottimizzazioni importanti che hanno portato a uno speedup di **947.34** rispetto alla prima versione sequenziale. Dato che il nostro problema è compute-bound, lo speedup è stato significativo in questa fase.
- **Dimensionamento blocchi:** abbiamo fatto vari test per capire quale fosse il modo conveniente di dimensionare i kernel di DFT, IDFT e filtro. La configurazione ottimale è risultato essere 16x8 per DFT e IDFT e 16x16 per il filtro.
- **Shared Memory:** ottimizzazione per favorire la collaborazione tra i thread che ha portato a uno speedup up di **1088.27** rispetto alla prima versione sequenziale.
- **Loop unrolling:** in questa fase le prestazioni sono notevolmente migliore, in quanto con unrolling 16 siamo riusciti ad arrivare al nostro speedup massimo rispetto alla versione sequenziale ovvero **2036.32**.
- **Cache L1:** l'ultima ottimizzazione che non ha portato miglioramenti nella velocità di esecuzione.

## 10 Estensione: Video Edge Detection

### 10.1 Intro

Il naturale passo successivo all'elaborazione di una singola immagine statica nel dominio delle frequenze è l'estensione dell'algoritmo a un flusso video, ovvero una sequenza temporale di fotogrammi. Questa transizione introduce sfide computazionali significativamente più complesse, guidate principalmente dalla necessità di garantire un throughput elevato e di minimizzare i tempi di latenza per approssimare, laddove possibile, i vincoli del tempo reale (real-time processing).

Mentre l'applicazione della DFT su un singolo frame richiede l'ottimizzazione del parallelismo massivo all'interno della GPU (parallelismo spaziale), l'elaborazione di un video impone una gestione oculata e strategica del flusso dei dati tra l'host (CPU) e il device (GPU) nel corso del tempo (parallelismo temporale). In contesti ad alto throughput, infatti, il collo di bottiglia principale è quasi sempre rappresentato dal trasferimento dei dati attraverso il bus PCIe: se gestito in modo puramente sequenziale (sincrono), il tempo speso per copiare i pixel dei frame rischia di lasciare i core di calcolo della GPU inutilizzati (idle), vanificando i vantaggi prestazionali dell'architettura parallela.

### 10.2 Implementazione naive

Il primo approccio all'elaborazione video ha richiesto la suddivisione in frame di un video di partenza, l'estrazione frame è stata fatta usando **ffmpeg**, i frame vengono salvati come immagini singole in formato pgm con dimensioni 960x540 pixel, per mantenere una linea coerente con il codice di base. Il numero totale di immagini ottenute è 818.

La prima implementazione, definita naive (ed evidente nel codice `cudaVideo.cu`), adotta una logica di esecuzione puramente sequenziale. All'interno del ciclo `for` principale, l'host esegue in ordine rigidamente seriale le seguenti operazioni per ciascun fotogramma:

1. Lettura sincrona del file PGM da disco tramite I/O di sistema
2. Trasferimento sincrono dei pixel dalla memoria host a quella device tramite *cudaMemcpy*
3. Invocazione in serie dei tre kernel sullo stream predefinito (Stream 0): calcolo della trasformata, applicazione del filtro passa-alto e computazione dell'antitrasformata
4. Sincronizzazione esplicita tramite la barriera *cudaDeviceSynchronize()*
5. Trasferimento sincrono del risultato dal device all'host tramite *cudaMemcpy*
6. Scrittura sincrona del frame risultante su disco

I tempi di esecuzione di questa architettura risultano estremamente elevati, pari a **2380.39** secondi. L'analisi del comportamento dell'applicazione effettuata tramite lo strumento di profiling NVIDIA Nsight Systems (nsys) ha permesso di mappare con precisione i tre colli di bottiglia principali che affliggono questa implementazione:

- **Serializzazione assoluta sul Default Stream:** Poiché tutte le operazioni vengono accodate sullo stream predefinito, la GPU non è in grado di sovrapporre i trasferimenti sul bus PCIe con l'esecuzione dei kernel. Di conseguenza, mentre la memoria viene copiata, i core di calcolo rimangono inattivi (idle), e viceversa, impedendo il parallelismo temporale.
- **Impatto sostanziale della barriera di sincronizzazione:** L'inserimento dell'istruzione `cudaDeviceSynchronize()` interrompe forzatamente il thread della CPU, costringendo l'host ad attendere passivamente il completamento delle attività sulla GPU. Questo annulla l'asincronia nativa di CUDA: la CPU non può sfruttare il tempo in cui la GPU calcola per avviare la lettura del fotogramma successivo o la scrittura di quello precedente.
- **Overhead di I/O su disco (CPU-Bound):** Le funzioni di lettura e scrittura su file system operano in modo bloccante. Poiché il ciclo è strettamente seriale, il tempo speso dalla CPU per accedere al disco si somma direttamente al tempo di computazione della GPU. Dato che i tempi di accesso al disco rigido o all'unità di storage sono di ordini di grandezza superiori rispetto al calcolo in memoria volatile, la GPU trascorre la stragrande maggioranza del tempo totale in uno stato di inattività, in attesa che l'host le fornisca il blocco di dati successivo.

Questa analisi mette in luce come il limite prestazionale critico del sistema non risieda nell'efficienza computazionale dei singoli kernel della GPU, bensì nell'architettura sincrona della pipeline di sdoppiamento e gestione dei dati. Quanto detto risulta visibile nell'immagine seguente:

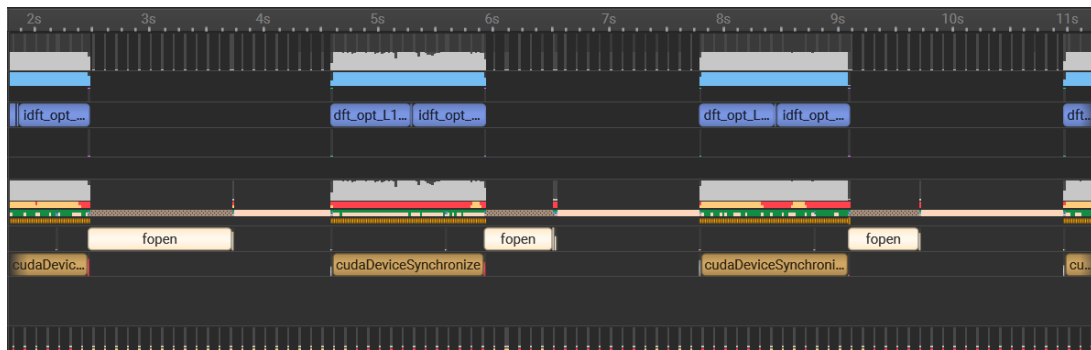


Figura 27: Porzione di flusso di esecuzione su video di `cudaVideo.cu`

### 10.3 Ottimizzazione tramite CUDA Streams e Kernel Fusion

Il primo importante passo verso l'ottimizzazione prestazionale del flusso video ha previsto una duplice strategia architetturale: la fusione dei kernel computazionali (*kernel fusion*) e l'introduzione del parallelismo temporale tramite i *CUDA Streams*.

In primo luogo, si è proceduto alla fusione del codice contenuto nel kernel *filtro* direttamente all'interno del kernel di antitrasformata (*idft*). Questa operazione di *kernel fusion* apporta un duplice beneficio: da un lato riduce l'overhead legato al lancio sequenziale di molteplici kernel da parte dell'host; dall'altro, elimina la necessità di scrivere i risultati intermedi della DFT nella memoria globale del device per poi doverli rileggere nel kernel

Il fulcro di questa versione (sviluppata in *optVideo1.cu*) risiede tuttavia nell'adozione di una pipeline asincrona basata su due *CUDA Streams* gestiti in modalità *double buffering* (`stream[0]` e `stream[1]`). A differenza della versione naive, in cui l'intero sistema operava in modo sincrono e sequenziale, lo schema multi-stream consente di sovrapporre temporalmente le operazioni di I/O della CPU, i trasferimenti di memoria sul bus PCIe e il calcolo parallelo sulla GPU.

1. Mentre lo stream `cur` presiede all'elaborazione dei kernel sulla GPU, la CPU si porta avanti eseguendo la lettura asincrona dal disco del fotogramma successivo ( $i + 1$ ) tramite la funzione `readPGM`.
2. Immediatamente dopo, i pixel del nuovo frame vengono trasferiti dall'host al device in modalità non bloccante tramite la funzione `cudaMemcpyAsync` associata allo `stream[next]`.
3. Contemporaneamente, i risultati già calcolati dal fotogramma corrente vengono scaricati in memoria host (`DeviceToHost`) sempre in modo asincrono tramite lo `stream[cur]`.
4. Prima di procedere alla fase di scrittura del file su disco (operazione strettamente CPU-bound), l'host invoca la funzione `cudaStreamSynchronize(stream[cur])`. Questa barriera è estremamente granulare: essa arresta la CPU solo fino al completamento delle attività dello stream corrente, lasciando l'altro stream libero di continuare a trasferire o elaborare i dati in background.

25

## 10.4 Pipelining Avanzato e Risoluzione del GPU Starvation

Nonostante l'introduzione degli stream e del *double buffering*, l'analisi approfondita della prima versione asincrona ha rivelato un limite architetturale residuo: la presenza di *starvation* della GPU. Nel ciclo iterativo iniziale, l'host accodava il trasferimento di memoria del frame successivo, ma invocava la barriera `cudaStreamSynchronize` prima di averne accodato i kernel computazionali. Di conseguenza, durante i prolungati tempi di latenza dovuti alla scrittura del file su disco rigido (`writePGM`), la coda dei comandi della GPU si svuotava prematuramente, costringendo il device ad attendere in stato di *idle* il risveglio della CPU.

Per risolvere definitivamente questo collo di bottiglia e massimizzare il parallelismo, nell'iterazione finale del progetto (implementata in `optVideo2.cu`) è stata sviluppata un'architettura di *pipelining* a due stadi pienamente sovrapposti. La chiave di volta di questa ottimizzazione risiede nel disaccoppiamento totale tra la sottomissione dei task e la loro sincronizzazione, ottenuto tramite un pre-caricamento aggressivo (*pre-fetching*).

Prima di avviare il ciclo principale, l'host accoda per intero le direttive dei primi due fotogrammi (copia *Host-to-Device*, calcolo della DFT, Edge Detection e copia *Device-to-Host*) sui rispettivi stream 0 e 1, riempiendo così la pipeline del *driver* NVIDIA. All'interno del loop iterativo, la logica di gestione viene conseguentemente ribaltata:

- L'host attende unicamente il completamento dello stream corrente (`cur`) relativo al frame  $i$  e procede alla sua scrittura su disco.
- **Latency Hiding:** Durante i millisecondi spesi dalla CPU per l'I/O bloccante su disco, la GPU non è più inattiva, bensì sta già processando a pieno carico il frame successivo ( $i+1$ ) sullo stream alternato (`next`), le cui istruzioni erano state accodate in precedenza.
- Immediatamente dopo la scrittura, la CPU legge da disco il fotogramma  $i+2$  e ne accoda l'intero ciclo vitale sullo stream `cur` appena liberatosi, mantenendo un flusso continuo di istruzioni verso il *device*.

I riscontri empirici generati dai report di *Nsight Systems* confermano il raggiungimento dell'efficienza architetturale massima per questa tipologia di algoritmo. I grafici di *timeline* evidenziano la scomparsa di qualsiasi *gap* (spazio vuoto) tra le esecuzioni dei kernel, dimostrando un perfetto *overlap* temporale. L'impatto dei trasferimenti sul bus PCIe e l'I/O del file system risultano interamente celati dietro al massiccio carico aritmetico della Trasformata di Fourier discreta. In questo scenario *\*compute-bound\**, il tasso di occupazione della GPU tocca costantemente picchi superiori al 99.9%, segno che le risorse hardware disponibili vengono sfruttate fino al loro limite teorico.

Il tempo di esecuzione di questa versione di attesta sui **1151.58** secondi ed è possibile notare tutto questo nell'immagine seguente:

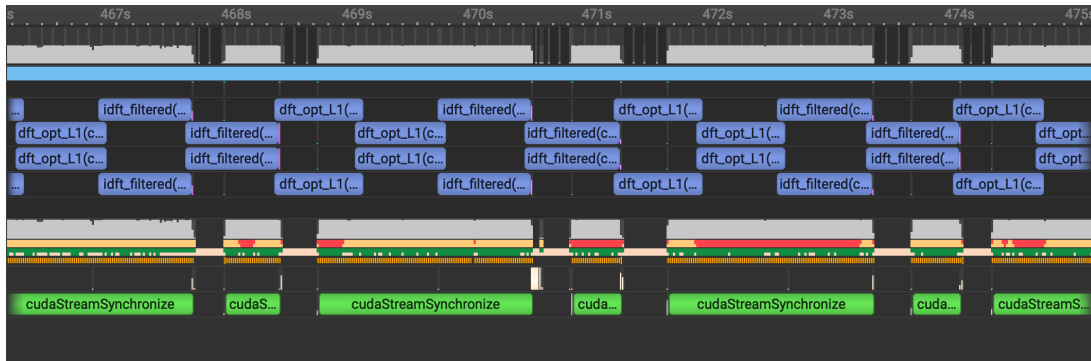


Figura 29: Porzione di flusso di esecuzione su video di optVideo2.cu

## 10.5 Risultati

Di seguito sono mostrati i tempi di esecuzione delle 3 versioni dell'estensione realizzata per Edge Detection su video e gli speedup delle due ottimizzazioni rispetto alla versione Naive:

|              | Tempo(s) | Speedup |
|--------------|----------|---------|
| <b>Naive</b> | 2380.39  | -       |
| <b>Opt1</b>  | 1200.65  | 1.98    |
| <b>Opt2</b>  | 1151.58  | 2.07    |

Tabella 11: Risultati Video Edge Detection